ICON

Industrial Diagnosis

Sistema intelligente per la diagnosi simbolica e la valutazione ML dei guasti industriali

Autore: Antonio De Rosa

Corso: Ingegneria della Conoscenza

Link GitHub: ICON-25-26

Anno accademico: 2025-2026

Relazione tecnica del progetto

Sommario

Capitolo 1 – Introduzione

1.1 Dominio di interesse

1.2 Sommario del KBS

1.3 Requisiti funzionali e strumenti

1.4 Argomenti di interesse

1.5 Corrispondenza con il programma e delimitazione del progetto

Capitolo 2 – Rappresentazione della Conoscenza (Ontologie)

2.1 Ingegnerizzazione dell'Ontologia e Scelte Implementative

2.2 Definizione della TBox: Tassonomia delle Classi e Gerarchie

2.3 Object Properties: Relazioni, Domini e Codomini

2.4 Restrizioni Logiche ed Assiomi (Description Logic)

2.5 Definizione della ABox: Istanze, Topologia e Sintomi Grezzi

2.6 Dimensioni e complessità della Knowledge Base

Capitolo 3 – Ragionamento Automatico

3.1 Il Paradigma del Ragionamento Automatico nel KBS

3.2 Il Motore di Inferenza HermiT e le Logiche Descrittive

3.3 Implementazione del Modulo semantic_reasoner.py

3.4 Meccanismo di Materializzazione dei Sintomi ed Estrazione Classi Indirette

3.5 Ottimizzazione delle Prestazioni: Il Sistema di Caching Semantico

Capitolo 4 – Apprendimento e Incertezza (OntoBK)

4.1 Incertezza nel Dominio e Modellazione Multimodale dei Dati

4.2 Il Paradigma OntoBK e il Semantic Lifting

4.3 Pre-processing dello Spazio delle Feature: Standardizzazione e Codifica Nominale

4.4 Il Modello Predittivo Ensemble: Random Forest

4.5 Classificazione a Massimizzazione del Margine: Support Vector Machines (SVM)

4.6 Il Paradigma Connessionista: Reti Neurali Artificiali (Multi-Layer Perceptron)

4.7 Valutazione Statistica Rigorosa e Analisi Comparativa delle Prestazioni

4.8 Integrazione tra supporto simbolico e classificazione ML

Capitolo 5 – Conclusioni e Sviluppi Futuri

5.1 Sintesi dei Risultati Raggiunti

5.2 Valutazione Critica dell'Architettura

5.3 Sviluppi Futuri

Capitolo 1 – Introduzione

1.1 Dominio di interesse

Il progetto ICON — Industrial Diagnosis affronta il problema della diagnosi preliminare dei guasti in sistemi industriali. In un impianto reale, un'anomalia può manifestarsi attraverso misure numeriche (ad esempio temperatura, corrente, vibrazioni, pressione e portata) e attraverso sintomi descritti in forma categoriale. La stessa evidenza può essere compatibile con più guasti; inoltre, non tutti i sensori sono disponibili su ogni macchina e alcune osservazioni possono essere sconosciute o non applicabili.

Il progetto ICON — Industrial Diagnosis affronta il problema della diagnosi preliminare dei guasti in sistemi industriali. Un guasto raramente è identificabile da una sola osservazione: la stessa anomalia può avere cause differenti e una causa può manifestarsi attraverso più sintomi. Il sistema rappresenta quindi il caso diagnostico mediante due tipi di informazione complementari: misure numeriche (per esempio temperatura, corrente, vibrazioni, pressione e portata) e osservazioni categoriali, come sintomi presenti, assenti o non osservabili. La disponibilità delle misure dipende anche dal tipo di macchina: pressione e portata, per esempio, sono pertinenti soprattutto alle macchine con circuito idraulico.

Il prototipo considera quattro tipologie di macchina: motore elettrico, pompa industriale, compressore e nastro trasportatore. Le ipotesi diagnostiche sono organizzate in quattro famiglie: guasto termico, meccanico, elettrico e idraulico. La compatibilità fra macchina e famiglia di guasto viene rappresentata nella base di conoscenza e utilizzata nella diagnosi simbolica per non proporre come pertinenti ipotesi incompatibili con la macchina selezionata. Il dominio è volutamente circoscritto per rendere espliciti e verificabili i collegamenti fra sintomi, componenti e guasti.

1.2 Sommario del KBS

Il Knowledge-Based System (KBS) integra una base di conoscenza OWL, un motore di inferenza e una componente di apprendimento automatico. Le due modalità sono complementari, ma non vanno confuse: la diagnosi simbolica produce un ordinamento di ipotesi in base al supporto delle evidenze e ai vincoli di compatibilità; la valutazione ML confronta classificatori addestrati su feature grezze e su una rappresentazione arricchita con informazioni semantiche.

1. Il Knowledge-Based System (KBS) integra una base di conoscenza OWL, un motore di inferenza e una componente di apprendimento automatico supervisionato. I componenti cooperano ma svolgono compiti distinti. L'ontologia formalizza il vocabolario del dominio e le relazioni fra le entità; HermiT deduce appartenenze semantiche implicite a partire dagli assiomi; la diagnosi simbolica ordina le ipotesi sulla base delle evidenze e della compatibilità della macchina; i classificatori ML apprendono invece correlazioni dai casi del dataset. Il confronto Base/OntoBK serve a misurare se l'aggiunta di feature derivate dalla conoscenza ontologica apporti un vantaggio predittivo rispetto alle feature originarie.
2. Esecuzione del reasoner HermiT per classificare semanticamente gli individui in base agli assiomi OWL.
3. Acquisizione di una macchina e dello stato dei sintomi nella diagnosi simbolica.
4. Calcolo del supporto simbolico per i guasti compatibili, distinguendo evidenze favorevoli, sintomi associati ma dichiarati assenti e stati ignorati.
5. Calcolo del supporto simbolico per i guasti compatibili, separando i sintomi favorevoli, i sintomi associati al guasto ma dichiarati assenti e gli stati che non costituiscono evidenza.

1.3 Requisiti funzionali e strumenti

Requisito	Realizzazione
R1 — Modellazione della conoscenza	Ontologia OWL costruita in Python tramite Owlready2.
R2 — Inferenza automatica	Reasoner HermiT invocato tramite sync_reasoner.
R3 — Gestione delle evidenze	Stati presente, assente, sconosciuto e non applicabile.
R4 — Diagnosi simbolica	Filtraggio per compatibilità macchina-guasto e conteggio di sintomi di supporto.
R5 — Apprendimento supervisionato	Pipeline scikit-learn con Random Forest, SVM RBF e MLP.
R6 — Confronto Base/OntoBK	Stessa procedura di validazione per le due rappresentazioni.
R7 — Riproducibilità	Dataset sintetico generabile con seed, risultati CSV e test automatici.

Gli strumenti principali sono Python, Owlready2, un'installazione Java compatibile con HermiT, pandas, NumPy e scikit-learn. I test sono realizzati con pytest. I file di output comprendono l'ontologia esportata, i risultati comparativi e le matrici di confusione. Il punto di ingresso è il modulo src/main.py, avviabile dalla directory principale con il comando seguente:

```
python -m src.main
```

1.4 Argomenti di interesse

- Rappresentazione simbolica della conoscenza tramite ontologie e Description Logics.
- Inferenza automatica e classificazione di individui a partire da assiomi espliciti.
- Gestione di informazione incompleta e distinzione fra evidenza assente e non osservata.
- Integrazione di Knowledge Base e Machine Learning attraverso Ontology-Based Knowledge (OntoBK).
- Valutazione empirica mediante cross-validation ripetuta e metriche macro per classificazione multiclasse.
- Analisi critica dei casi in cui l'arricchimento semantico non migliora le prestazioni predittive.

1.5 Corrispondenza con il programma e delimitazione del progetto

Il progetto si concentra sui temi del programma più direttamente pertinenti a un KBS con integrazione ML+OntoBK. Il collegamento principale è con il capitolo 16 (Knowledge Graph, ontologie, classi, proprietà e logiche descrittive), con il capitolo 7 (apprendimento supervisionato, classificazione, cross-validation e confronto di modelli) e con il capitolo 8 (rete neurale feed-forward MLP). Il capitolo 5 è richiamato nella rappresentazione simbolica delle evidenze e nella produzione di spiegazioni circoscritte; il capitolo 15 è pertinente alla struttura relazionale della conoscenza, pur senza implementare un interprete Datalog/Prolog autonomo.

Il progetto approfondisce tre nuclei del programma direttamente utilizzati nell'implementazione. Il primo è la rappresentazione della conoscenza mediante ontologie e logiche descrittive: classi, gerarchie, individui, proprietà e assiomi OWL permettono di descrivere in modo formale il dominio industriale. Il secondo è il ragionamento automatico: il reasoner HermiT classifica gli individui in base agli assiomi, rendendo disponibili informazioni semantiche non esplicitate direttamente. Il terzo è l'apprendimento supervisionato: Random Forest, SVM RBF e MLP vengono addestrati a predire la famiglia di guasto a partire da esempi etichettati e valutati con cross-validation.

Un ulteriore collegamento riguarda la gestione dell'informazione incompleta nella diagnosi simbolica. Gli stati PRESENT, ABSENT, UNKNOWN e NON_APPLICABLE hanno significati distinti: un sintomo assente può costituire evidenza contraria rispetto a un guasto che normalmente lo prevede, mentre uno stato sconosciuto o non applicabile non viene trattato come assenza. Questa distinzione riguarda la rappresentazione delle evidenze e non costituisce, da sola, un modello probabilistico dell'incertezza.

Il progetto segue in particolare la proposta ML+OntoBK: confronta classificatori supervisionati usando una rappresentazione Base e una rappresentazione arricchita con conoscenza ontologica. La scelta è sperimentale: l'utilità delle feature semantiche viene misurata e non assunta a priori. Il risultato del confronto è parte integrante del contributo, anche quando l'arricchimento non migliora le metriche.

Capitolo 2 – Rappresentazione della Conoscenza (Ontologie)

2.1 Ingegnerizzazione dell'Ontologia e Scelte Implementative

L'ontologia viene costruita programmaticamente in src/ontology.py attraverso Owlready2. La scelta permette di mantenere gli assiomi vicino al codice, ricostruire la base di conoscenza in modo riproducibile ed esportarla in formato RDF/XML. L'IRI configurato nel prototipo è https://example.org/icon/industrial-diagnosis.owl: è un identificatore dimostrativo, da sostituire con un IRI stabile se l'ontologia verrà pubblicata come risorsa web.

Il modello separa concetti di dominio (classi), relazioni fra individui (object properties) e attributi letterali delle osservazioni (data properties). La separazione rende possibile esprimere vincoli semantici indipendenti dalla logica procedurale del programma Python.

Elemento	Scopo
Machine / MachineType	Rappresenta una macchina e i suoi tipi.
Component	Rappresenta componenti fisici rilevanti per la diagnosi.
Sensor	Rappresenta sensori associati alle macchine e ai componenti monitorati.
Observation	Classe predisposta per rappresentare una rilevazione; nella versione corrente non sono presenti individui Observation operativi.
Symptom	Rappresenta un sintomo diagnostico osservabile.
Fault / FaultFamily	Organizza le ipotesi diagnostiche per famiglia.

2.2 Definizione della TBox: Tassonomia delle Classi e Gerarchie

La TBox contiene le classi e gli assiomi che definiscono il vocabolario del dominio. La gerarchia principale distingue Machine, Component, Sensor, Observation, Symptom e Fault. FaultFamily è una sottoclasse di Fault e si specializza in ThermalFault, MechanicalFault, ElectricalFault e HydraulicFault. MachineType specializza Machine e comprende ElectricMotor, IndustrialPump, Compressor e ConveyorBelt.

Sono inoltre definite sottoclassi per categorie di sensori (TemperatureSensor, CurrentSensor, VibrationSensor, PressureSensor e FlowSensor) e per componenti (MotorComponent, Bearing, CoolingSystem, ElectricalSupply, HydraulicCircuit e BeltDrive). Le classi sintomatiche ThermalSymptom, MechanicalSymptom, ElectricalSymptom e HydraulicSymptom vengono definite attraverso assiomi di equivalenza, in modo da poter essere riconosciute dal reasoner sulla base delle relazioni esplicite.

La gerarchia è volutamente compatta: il progetto dimostra come usare tassonomia e assiomi per derivare classificazioni, senza pretendere di rappresentare tutte le componenti e modalità di guasto presenti in un impianto industriale complesso.

2.3 Object Properties: Relazioni, Domini e Codomini

Le object properties collegano gli individui del dominio. Le principali relazioni utilizzate nel modello e rilevanti per il flusso operativo sono riportate nella tabella. Nel codice non sono dichiarate proprietà transitive o inverse: pertanto la relazione non va descritta come transitiva né si deve presumere che il reasoner generi automaticamente relazioni inverse non assiomatizzate. Il ragionamento deriva invece dalle gerarchie e dagli assiomi effettivamente dichiarati.

Proprietà	Dominio → Codominio	Significato
has_component	Machine → Component	La macchina include un componente.
has_sensor	Machine → Sensor	Il sensore è associato alla macchina.
monitors_component	Sensor → Component	Il sensore monitora un componente.
indicates_fault	Symptom → FaultFamily	Il sintomo è associato a una famiglia di guasto.
affects_component	FaultFamily → Component	La famiglia di guasto interessa un componente.
compatible_with_machine	FaultFamily → MachineType	La famiglia è compatibile con un tipo di macchina.

Le object properties collegano gli individui che compongono il modello del dominio. Le relazioni più rilevanti per il flusso corrente descrivono la composizione della macchina, l'associazione dei sensori ai componenti, il legame fra sintomi e famiglie di guasto, i componenti interessati e la compatibilità fra famiglia di guasto e tipo di macchina. Domain e range dichiarano quali tipi di entità partecipano a ciascuna relazione e possono contribuire alla classificazione inferenziale OWL.

Le data properties observation_state, measurement_value e measurement_unit descrivono rispettivamente lo stato dell'osservazione, il valore numerico e l'unità di misura. Sono dichiarate funzionali, cioè ciascuna osservazione ha al più un valore per la specifica proprietà funzionale.

Nel flusso operativo la parte più importante della rete di relazioni è il collegamento Symptom → FaultFamily tramite indicates_fault e il collegamento FaultFamily → MachineType tramite compatible_with_machine. Il primo fornisce la base semantica per classificare i sintomi; il secondo permette al ragionamento diagnostico di applicare i vincoli di compatibilità prima di ordinare le ipotesi.

Le definizioni equivalenti dei sintomi usano una restrizione esistenziale. Per esempio, ThermalSymptom è equivalente all'intersezione fra Symptom e l'insieme degli individui che sono collegati tramite indicates_fault ad almeno un individuo di ThermalFault. In notazione descrittiva:

2.4 Restrizioni Logiche ed Assiomi (Description Logic)

Le definizioni equivalenti dei sintomi usano una restrizione esistenziale, uno dei costrutti delle logiche descrittive. Per esempio, ThermalSymptom è definita come l'intersezione fra la classe Symptom e gli individui collegati, tramite indicates_fault, ad almeno un individuo di ThermalFault. La notazione è:

Lo stesso schema viene applicato alle famiglie meccanica, elettrica e idraulica. In presenza degli assiomi e delle relazioni necessarie, HermiT può inferire che un individuo sintomo appartenga alla corrispondente classe specializzata anche se tale appartenenza non è stata asserita direttamente.

Lo stesso schema viene applicato alle famiglie meccanica, elettrica e idraulica. Se un individuo appartiene a Symptom ed è collegato tramite indicates_fault a un individuo di ThermalFault, l'assioma consente al reasoner di inferire che esso appartenga anche a ThermalSymptom, senza che tale appartenenza debba essere inserita manualmente. È un esempio di classificazione automatica guidata dalla definizione di classe, anziché da un elenco di etichette scritto a mano.

2.5 Definizione della ABox: Istanze, Topologia e Sintomi Grezzi

La ABox comprende individui concreti che popolano le classi. Nel modello sono presenti individui rappresentativi delle quattro famiglie di guasto, dei quattro tipi di macchina, dei componenti, dei sensori e dei sintomi. Fra i sintomi figurano overheating, abnormal_vibration, current_anomaly, pressure_anomaly, flow_anomaly, unusual_noise e reduced_performance. Le classi Observation, le proprietà has_observation, observes_symptom e concerns_machine e le data properties observation_state, measurement_value e measurement_unit sono definite

nello schema ontologico, ma nella versione corrente non sono popolate da individui ABox e non costituiscono il percorso operativo usato dalla diagnosi o dall'estrazione ML; rappresentano quindi una struttura predisposta per estensioni future.

La ABox contiene individui rappresentativi delle quattro famiglie di guasto, dei quattro tipi di macchina, dei componenti, dei sensori e dei sintomi. Fra i sintomi modellati figurano overheating, abnormal_vibration, current_anomaly, pressure_anomaly, flow_anomaly, unusual_noise e reduced_performance. Le asserzioni collegano macchine e componenti, sensori e componenti monitorati, sintomi e famiglie di guasto, oltre alle compatibilità macchina-guasto.

Queste asserzioni costituiscono la rete di conoscenza generale su cui opera HermiT. Il dataset tabellare contiene invece i singoli casi sintetici e i rispettivi stati delle evidenze: ogni riga non viene trasformata automaticamente in un individuo OWL distinto durante la diagnosi simbolica. La distinzione è intenzionale: l'ontologia codifica relazioni e categorie riutilizzabili, mentre il dataset fornisce esempi etichettati per l'apprendimento supervisionato.

Il collegamento fra i due livelli avviene attraverso il modulo di estrazione delle feature, che legge dall'ontologia le classi inferite e le relazioni rilevanti e le converte in attributi numerici utilizzabili dai classificatori. In questo modo la conoscenza simbolica non sostituisce i dati: ne produce una rappresentazione aggiuntiva che può essere confrontata sperimentalmente con la rappresentazione Base.

2.6 Dimensioni e complessità della Knowledge Base

La complessità della KB viene valutata qui in termini strutturali e di copertura, non come complessità computazionale formale completa. Lo schema implementato dichiara 9 object properties e 3 data properties funzionali; include 4 assiomi di equivalenza per classificare i sintomi nelle rispettive famiglie semantiche e individui rappresentativi di 4 famiglie di guasto, 4 tipi di macchina, 6 componenti, 5 sensori e 7 sintomi. Le relazioni collegano macchine, componenti, sensori, sintomi e famiglie di guasto; le compatibilità macchina-guasto restringono le ipotesi ammissibili durante la diagnosi.

La base di conoscenza comprende 9 object properties e 3 data properties funzionali nello schema, quattro assiomi di equivalenza per classificare i sintomi nelle rispettive famiglie semantiche e individui rappresentativi di quattro famiglie di guasto, quattro tipi di macchina, sei componenti, cinque sensori e sette sintomi. Queste dimensioni descrivono la versione didattica documentata e possono essere verificate nel modulo src/ontology.py.

Dal punto di vista strutturale, la KB è compatta e leggibile: la parte inferenziale più significativa deriva dagli assiomi OWL e dalle relazioni asserite; il ranking diagnostico è invece una procedura separata che conteggia le evidenze e filtra le incompatibilità. Per un caso singolo, il ranking esamina un numero finito di sintomi e famiglie di guasto, quindi il suo costo cresce con le dimensioni di questi insiemi. Questa osservazione riguarda la procedura di ranking e non sostituisce un'analisi formale della complessità del reasoner OWL.

La KB rappresenta un modello didattico circoscritto e non una tassonomia esaustiva dei guasti industriali. Il suo obiettivo è mostrare come una conoscenza esplicita possa essere strutturata, interrogata e collegata a una fase di apprendimento, mantenendo distinguibili il livello simbolico e quello statistico.

Capitolo 3 – Ragionamento Automatico

3.1 Il Paradigma del Ragionamento Automatico nel KBS

Il ragionamento automatico consente di ricavare conoscenza implicita dagli assiomi della base. Nel progetto il reasoner OWL classifica gli individui sintomo e rende disponibili le classi inferite. A valle di tale inferenza, la diagnosi simbolica applica una procedura esplicita di confronto delle evidenze: non è un calcolo probabilistico e non restituisce una probabilità calibrata del guasto.

Il ragionamento automatico consente di ricavare conoscenza implicita dagli assiomi della base. Nel progetto HermiT classifica gli individui sintomo e rende disponibili le classi inferite; la diagnosi simbolica usa poi queste relazioni insieme agli stati inseriti dall'utente. È importante distinguere i due processi: il reasoner esegue inferenza OWL, mentre il ranking diagnostico è un algoritmo procedurale basato sulle evidenze e non un calcolo probabilistico.

Il flusso diagnostico segue questi passaggi: (1) viene selezionato il tipo di macchina; (2) l'utente assegna agli indicatori uno stato osservativo; (3) il sistema considera soltanto le ipotesi compatibili con la macchina; (4) per ogni ipotesi conta i sintomi presenti che risultano associati al guasto; (5) evidenzia separatamente gli eventuali sintomi associati ma dichiarati assenti; (6) ordina le ipotesi in base al supporto raccolto e restituisce gli elementi che giustificano il risultato. Gli stati sconosciuto e non applicabile non aumentano né diminuiscono il supporto.

3.2 Il Motore di Inferenza HermiT e le Logiche Descrittive

HermiT è un reasoner OWL 2 basato su logiche descrittive. Nel progetto viene invocato tramite owlready2.sync_reasoner sul mondo OWL creato da build_ontology(). L'opzione infer_property_values=True richiede anche l'inferenza di valori di proprietà supportati dal reasoner. Le classificazioni risultanti sono poi interrogate attraverso INDIRECT_is_a, che permette di accedere alle appartenenze dirette e indirette dell'individuo.

L'esecuzione del reasoner è necessaria prima di interrogare le classi inferite. Il modulo controlla tale prerequisito tramite reasoner_run e solleva un errore se si tenta di accedere alle classi senza aver eseguito l'inferenza. Questo rende esplicito il ciclo di vita del componente e riduce il rischio di utilizzare una cache non inizializzata.

3.3 Implementazione del Modulo semantic_reasoner.py

DiagnosticReasoner incapsula la gestione del reasoner e la procedura di diagnosi simbolica. Le sue responsabilità principali sono l'esecuzione dell'inferenza, il recupero degli individui OWL, l'estrazione delle classi inferite e l'elaborazione degli stati dei sintomi.

- FAULTS mappa le etichette del dataset ai nomi degli individui OWL delle famiglie di guasto.
- SYMPTOMS definisce il vocabolario dei sintomi riconosciuti dal sistema.
- VALID_STATES documenta gli stati ammessi dalla diagnosi simbolica: PRESENT, ABSENT, UNKNOWN e NON_APPLICABLE.
- get_inferred_classes_for_symptom controlla l'avvenuta inferenza, verifica che l'individuo sia un sintomo ed estrae le classi indirette pertinenti.
- diagnose applica la compatibilità macchina-guasto, raccoglie le evidenze favorevoli e segnala i sintomi associati dichiarati assenti.

Il supporto di una diagnosi è un conteggio simbolico. Non deve essere interpretato come confidenza statistica: due guasti con supporto 3 non hanno necessariamente probabilità del 50%, e il conteggio non considera esplicitamente la frequenza empirica dei guasti né la dipendenza fra sintomi.

Il supporto diagnostico è un conteggio simbolico e va interpretato come indicatore di coerenza con le evidenze codificate, non come probabilità o confidenza statistica. Per esempio, un supporto pari a 3 significa che tre sintomi presenti contribuiscono a sostenere l'ipotesi secondo le relazioni della KB; non significa che il guasto abbia probabilità del 75% o che sia certo. Il ranking non stima la frequenza dei guasti e non modella esplicitamente la dipendenza fra sintomi.

3.4 Meccanismo di Materializzazione dei Sintomi ed Estrazione Classi Indirette

Gli assiomi di equivalenza permettono di classificare indirettamente gli individui sintomo. Per esempio, se un sintomo è collegato tramite indicates_fault a un individuo di ThermalFault, la definizione di ThermalSymptom rende possibile inferire l'appartenenza alla classe. L'estrazione avviene interrogando le classi indirette dell'individuo e rimuovendo le classi generiche escluse (Thing, Symptom, Fault e FaultFamily).

La materializzazione è qui intesa come disponibilità delle classificazioni inferite nell'ambiente OWL dopo l'esecuzione del reasoner. Nel controllo eseguito sul file esportato, le classi specifiche inferite per gli individui sintomo verificati risultano ancora presenti dopo il caricamento da disco. Non si tratta di un processo di apprendimento di nuovi assiomi né di aggiornamento automatico della conoscenza a partire dai risultati ML.

3.5 Ottimizzazione delle Prestazioni: Il Sistema di Caching Semantico

DiagnosticReasoner mantiene cache_inferenze, un dizionario che associa il nome di ciascun sintomo alle classi inferite già estratte. Dopo una nuova esecuzione di run_inference(), la cache viene svuotata per evitare di riutilizzare risultati potenzialmente non più validi. Se la richiesta riguarda un sintomo già presente nella cache, il modulo restituisce una copia della lista memorizzata invece di ripetere la ricerca e l'estrazione.

La cache riduce il lavoro ripetuto nelle chiamate successive, soprattutto quando la stessa ontologia viene utilizzata per più casi o durante l'estrazione delle feature semantiche. Non sostituisce HermiT e non evita il costo della prima inferenza: ottimizza le interrogazioni ripetute successive.

Capitolo 4 – Apprendimento e Incertezza (OntoBK)

4.1 Incertezza nel Dominio e Modellazione Multimodale dei Dati

La diagnosi industriale è caratterizzata da misure numeriche variabili, sensori mancanti, osservazioni rumorose e sintomi non sempre disponibili. Il dataset industrial_faults.csv contiene 3.000 casi sintetici e 22 colonne, compresa la variabile target fault. Le variabili descrittive comprendono tipo di macchina, frazione di carico, ore operative, misure di temperatura/corrente/vibrazione/pressione/portata, stati dei sensori e stati dei sintomi. La colonna case_id identifica il caso ed è conservata per tracciabilità, ma viene esclusa dalle feature passate ai modelli.

La generazione usa valori di riferimento dipendenti dal tipo di macchina, effetti illustrativi per ciascuna famiglia di guasto, rumore casuale e probabilità configurate per i sintomi. Gli stati sconosciuti e le misure mancanti sono introdotti con probabilità impostabili. La generazione con random_state fisso rende ripetibile la costruzione del dataset.

Variabile / gruppo	Descrizione
machine_type	Motore elettrico, pompa industriale, compressore, nastro trasportatore.
load_fraction, operating_hours	Carico relativo e ore operative.
temperature_c, current_a, vibration_mm_s	Misure numeriche di temperatura, corrente e vibrazione.
pressure_bar, flow_rate_l_min	Pressione e portata; non applicabili a tutte le macchine.
*_status	Stato di disponibilità o mancanza del sensore.
symptom_*	Stato dei sette sintomi: presente, assente, sconosciuto o non applicabile.
fault	Classe target a quattro valori.

Il dataset è costruito per rendere possibile il confronto metodologico. Poiché la generazione usa effetti e probabilità specificati a priori, i risultati misurano quanto i modelli recuperino i pattern simulati e non dimostrano direttamente prestazioni su impianti reali.

4.2 Il Paradigma OntoBK e il Semantic Lifting

L'approccio OntoBK arricchisce la rappresentazione tabellare con feature derivate dalla base di conoscenza. La rappresentazione Base include feature numeriche, codifica del tipo di macchina e codifica degli stati dei sensori e dei sintomi. La rappresentazione OntoBK conserva le feature Base e aggiunge informazioni semantiche. Il SemanticFeatureExtractor viene adattato separatamente sui dati di training di ciascun fold e poi applicato alla relativa partizione di validazione: il vocabolario semantico non viene quindi appreso utilizzando i casi di validazione.

L'approccio OntoBK arricchisce la rappresentazione tabellare con feature derivate dalla base di conoscenza. La rappresentazione Base contiene le misure numeriche, la codifica del tipo di macchina e gli stati dei sensori e dei sintomi. La rappresentazione OntoBK conserva queste informazioni e aggiunge attributi semantici calcolati a partire dalle relazioni e dalle classi inferite. Il confronto è quindi un esperimento di ablazione: si mantiene il problema predittivo e si rimuove soltanto il gruppo di feature OntoBK per osservare quale contributo aggiuntivo produca.

SemanticFeatureExtractor recupera le classi semantiche inferite per i sintomi e le relazioni fra sintomi, famiglie di guasto e compatibilità con la macchina. Le feature aggiuntive rappresentano, tra l'altro, la presenza di categorie semantiche inferite, il numero di sintomi presenti associati a ciascuna famiglia, gli eventuali sintomi associati ma dichiarati assenti e la compatibilità dell'ipotesi con il tipo di macchina. Il vantaggio potenziale è rendere esplicite relazioni di dominio che il classificatore altrimenti dovrebbe ricostruire dai soli esempi; il rischio è aggiungere attributi ridondanti o poco discriminanti.

4.3 Pre-processing dello Spazio delle Feature: Standardizzazione e Codifica Nominale

Le feature numeriche vengono convertite in valori numerici e i valori non interpretabili diventano mancanti. Le categorie nominali sono codificate mediante indicatori one-hot: una colonna per ciascun tipo di macchina e per ciascuno stato ammesso dei sensori e dei sintomi. I valori mancanti o non riconosciuti negli stati vengono normalizzati nello stato SCONOSCIUTO, secondo le regole del trasformatore.

La pipeline applica l'imputazione mediana ai valori numerici mancanti e la standardizzazione delle feature. Standardizzare significa trasformare ogni variabile usando media e deviazione standard calcolate sui dati di addestramento, così che le scale delle misure risultino confrontabili. La trasformazione viene appresa all'interno di ciascun fold e poi applicata alla relativa partizione di validazione: in questo modo le statistiche del preprocessing non vengono stimate usando i dati di validazione. La standardizzazione è particolarmente importante per SVM e MLP, sensibili alla scala delle variabili; la stessa pipeline è mantenuta nel confronto per garantire condizioni coerenti.

4.4 Il Modello Predittivo Ensemble: Random Forest

Random Forest combina le predizioni di un insieme di alberi decisionali addestrati su campioni e sottoinsiemi di feature. La combinazione tende a ridurre la varianza rispetto a un singolo albero e può modellare relazioni non lineari fra misure e sintomi. La configurazione del progetto usa 200 alberi e imposta class_weight="balanced". Sebbene il dataset abbia lo stesso numero di esempi per ciascuna classe target, questa opzione fa parte della configurazione effettivamente valutata. Il modello viene valutato sia sulle feature Base sia sulle feature OntoBK, mantenendo uguale la procedura sperimentale.

Nei risultati registrati, Random Forest raggiunge accuracy 0,9032 con la rappresentazione Base e 0,8752 con OntoBK. In questo esperimento l'arricchimento semantico non migliora il modello: la differenza è di −2,80 punti percentuali. Il risultato evidenzia che feature semantiche ridondanti o non sufficientemente informative possono peggiorare le prestazioni, anche se sono concettualmente interpretabili.

4.5 Classificazione a Massimizzazione del Margine: Support Vector Machines (SVM)

La Support Vector Machine cerca una separazione a margine massimo fra classi; il kernel RBF permette di rappresentare confini non lineari nello spazio originale delle feature. La standardizzazione è importante perché le distanze utilizzate dal modello sono sensibili alla scala delle variabili. La configurazione usa class_weight="balanced", che assegna pesi alle classi durante l'addestramento; il dataset è bilanciato per costruzione, quindi l'opzione non risponde a uno squilibrio osservato nelle frequenze, ma viene mantenuta per descrivere fedelmente il modello implementato. La SVM è valutata nelle stesse condizioni dei modelli concorrenti.

L'accuracy osservata è 0,9084 per Base e 0,8981 per OntoBK, con una differenza di −1,03 punti percentuali. Anche in questo caso non emerge un vantaggio dell'arricchimento. Il dato va letto nel contesto del dataset sintetico e della specifica configurazione del modello, non come conclusione generale contro l'uso delle ontologie.

4.6 Il Paradigma Connessionista: Reti Neurali Artificiali (Multi-Layer Perceptron)

Il Multi-Layer Perceptron (MLP) è una rete neurale feed-forward che apprende trasformazioni non lineari tramite strati di neuroni e una funzione di perdita supervisionata. La configurazione implementata utilizza due strati nascosti di 50 e 25 unità e un limite di 500 iterazioni, con early stopping. Il pre-processing numerico aiuta l'ottimizzazione e rende le scale delle feature più confrontabili. I valori sono configurazioni sperimentali fissate, non il risultato di una ricerca esaustiva degli iperparametri: il seed 42 favorisce la riproducibilità, mentre le dimensioni (50, 25) e il limite di 500 iterazioni definiscono un modello di complessità moderata. L'early stopping limita

l'addestramento quando la prestazione di validazione interna non migliora. Anche per Random Forest (200 alberi) e SVM RBF vengono mantenute configurazioni dichiarate e riproducibili, senza affermare che siano ottimali.

L'MLP ottiene accuracy 0,9139 su Base e 0,9146 su OntoBK, la migliore coppia di risultati fra quelle registrate. Il vantaggio OntoBK è tuttavia molto piccolo: circa +0,067 punti percentuali. La differenza è insufficiente per affermare, senza ulteriori esperimenti e analisi d'incertezza, che l'arricchimento semantico migliori in modo sostanziale la generalizzazione.

4.7 Valutazione Statistica Rigorosa e Analisi Comparativa delle Prestazioni

Il confronto utilizza RepeatedStratifiedKFold con 5 fold e 3 ripetizioni, seed 42. In ogni ripetizione il dataset viene suddiviso in cinque parti, mantenendo per quanto possibile la proporzione delle classi: quattro parti vengono usate per addestrare e una per valutare, ripetendo il processo fino a usare ogni parte come validazione. La procedura viene ripetuta tre volte con partizioni differenti. Le due rappresentazioni Base e OntoBK sono valutate sugli stessi split, così la differenza osservata dipende più direttamente dalle feature utilizzate e meno dalla particolare suddivisione dei dati.

L'accuracy è la frazione di predizioni corrette sul totale. La precision macro calcola la precisione per ciascuna classe e ne fa la media; il recall macro fa lo stesso per la copertura dei casi reali di ciascuna classe; l'F1 macro calcola prima l'F1 di ogni classe e poi ne media i valori, attribuendo lo stesso peso a ogni famiglia di guasto. Le medie macro sono utili nella classificazione multiclasse perché impediscono alla classe più facile o frequente di dominare da sola la valutazione. La deviazione standard riassume la dispersione delle metriche fra i fold e le ripetizioni, ma non è di per sé un test di significatività statistica.

Modello	Rappresentazione	Accuracy ± std	F1 macro ± std	Δ accuracy OntoBK
Random Forest	Base	0,9032 ± 0,0142	0,9030 ± 0,0146	—
Random Forest	OntoBK	0,8752 ± 0,0142	0,8747 ± 0,0144	−2,80 p.p.
SVM RBF	Base	0,9084 ± 0,0100	0,9082 ± 0,0100	—
SVM RBF	OntoBK	0,8981 ± 0,0120	0,8978 ± 0,0121	−1,03 p.p.
MLP	Base	0,9139 ± 0,0126	0,9140 ± 0,0126	—
MLP	OntoBK	0,9146 ± 0,0098	0,9144 ± 0,0099	+0,067 p.p.

I risultati mostrano un quadro non uniforme. Base supera OntoBK per Random Forest e SVM; per MLP la versione OntoBK è lievemente migliore, ma il vantaggio è minimo rispetto alla variabilità osservata. Questo è un risultato utile: l'integrazione simbolico-statistica deve essere valutata sperimentalmente, non assunta come automaticamente benefica.

Le metriche riportate sono riferite al dataset sintetico generato dal progetto. Le deviazioni standard descrivono la variabilità delle valutazioni ripetute, ma non equivalgono a un intervallo di confidenza indipendente e non eliminano la necessità di una validazione esterna su dati reali. La ripetizione dei fold implica inoltre che le 9.000 predizioni aggregate non sono 9.000 casi indipendenti: rappresentano tre valutazioni ripetute degli stessi 3.000 casi.

4.8 Integrazione tra supporto simbolico e classificazione ML

La diagnosi simbolica e la classificazione ML rispondono a domande complementari. La prima usa relazioni esplicite della base di conoscenza e stati osservativi forniti dall'utente; la seconda apprende dai casi sintetici una funzione che associa le feature alla famiglia di guasto. Per questo i due output non devono essere interpretati come equivalenti: il supporto simbolico spiega quali relazioni ed evidenze sostengono un'ipotesi, mentre il classificatore restituisce la classe predetta e, se configurato per farlo, punteggi di classe che non sono automaticamente probabilità calibrate.

Nel percorso simbolico, l’interfaccia consente di selezionare il tipo di macchina e assegnare a ciascun sintomo uno stato: presente, assente, sconosciuto o non applicabile. Gli stati sconosciuti e non applicabili non vengono conteggiati come evidenza favorevole o contraria. I sintomi associati a un guasto ma dichiarati assenti sono riportati separatamente; le ipotesi incompatibili con la macchina sono escluse prima del ranking. Queste scelte rendono più leggibile il risultato e impediscono di interpretare automaticamente una mancata osservazione come prova dell’assenza del sintomo.

La spiegazione è limitata alle evidenze e alle relazioni codificate nella KB. Il sistema è un prototipo didattico di supporto alla diagnosi preliminare: il risultato deve essere verificato da personale competente e non sostituisce procedure di sicurezza o manutenzione certificate.

Capitolo 5 – Conclusioni e Sviluppi Futuri

5.1 Sintesi dei Risultati Raggiunti

Il progetto realizza un prototipo KBS che integra rappresentazione OWL, inferenza automatica, diagnosi simbolica e confronto di modelli supervisionati. L'ontologia descrive le categorie del dominio e le relazioni fra macchine, componenti, sensori, sintomi e famiglie di guasto. HermiT deduce appartenenze semantiche a partire dagli assiomi; la diagnosi simbolica usa compatibilità ed evidenze per ordinare le ipotesi; il modulo OntoBK genera una rappresentazione arricchita e la confronta con quella Base tramite modelli ML.

La valutazione registrata su 3.000 casi sintetici produce accuracy comprese fra circa 0,875 e 0,915 nelle configurazioni esaminate. L'MLP ottiene il valore più alto fra i risultati riportati, ma l'arricchimento semantico non produce un miglioramento uniforme: peggiora Random Forest e SVM e offre alla MLP un vantaggio molto contenuto. Il risultato sostiene una conclusione metodologica importante: l'utilità della conoscenza di fondo deve essere verificata sperimentalmente, non presunta in base alla sola presenza dell'ontologia.

5.2 Valutazione Critica dell'Architettura

I punti di forza sono la separazione fra conoscenza generale e dati tabellari, l'uso di un reasoner OWL, la spiegazione qualitativa del ranking simbolico, la gestione distinta degli stati osservativi, la generazione riproducibile del dataset e il confronto di più classificatori nelle stesse condizioni. I test automatici contribuiscono a verificare i comportamenti principali e i casi limite previsti dal codice.

I limiti principali sono la natura sintetica dei dati, la semplificazione delle famiglie di guasto, la copertura contenuta dell'ontologia e la natura euristica del ranking simbolico. Le distribuzioni del generatore possono rendere i pattern più regolari di quelli osservati in impianti reali; per questo i risultati misurano le prestazioni sullo scenario simulato e non costituiscono una stima diretta dell'efficacia industriale. Inoltre, le feature OntoBK non migliorano sistematicamente i classificatori nella configurazione sperimentata.

Il contributo del progetto consiste nell'aver costruito e valutato una pipeline in cui conoscenza esplicita e apprendimento statistico restano distinguibili e possono essere confrontati. La diagnosi simbolica favorisce la tracciabilità delle evidenze e dei vincoli di dominio; i modelli ML apprendono regolarità dal dataset. Il prototipo costituisce una base sperimentale didattica che richiede ulteriori verifiche prima di un eventuale impiego operativo.

5.3 Sviluppi Futuri

1. Sostituire o affiancare il dataset sintetico con dati industriali reali, documentati e raccolti in condizioni operative diversificate.

2. Estendere e validare con esperti le relazioni fra sintomi, componenti, famiglie di guasto e tipi di macchina, documentando le fonti della conoscenza inserita nell'ontologia.

3. Ampliare la valutazione con analisi per classe, esame degli errori e casi diagnostici rappresentativi, così da individuare quali guasti vengono confusi e in quali condizioni.

4. Approfondire l'ablation study delle feature semantiche e verificare se specifici gruppi di attributi OntoBK siano utili per alcuni modelli o famiglie di guasto.

5. Migliorare l'interfaccia con una visualizzazione del grafo della conoscenza e report diagnostici che mostrino le evidenze, le relazioni applicate e le ipotesi escluse.

Appendice A – Struttura essenziale del progetto

Percorso	Responsabilità
src/ontology.py	Costruzione OWL e riepilogo della base di conoscenza.
src/semantic_reasoner.py	Inferenza, cache semantica e diagnosi simbolica.
src/ontoBK_learning.py	Estrazione feature, pipeline ML e valutazione comparativa.
src/dataset_generator.py	Generazione riproducibile del dataset sintetico.
src/main.py	Interfaccia testuale e coordinamento delle funzioni.
data/industrial_faults.csv	Dataset sintetico utilizzato negli esperimenti.
results/industrial_diagnosis.owl	Esportazione dell'ontologia.
results/comparison_results.csv	Metriche aggregate dei confronti Base/OntoBK.
results/confusion_*.csv	Matrici di confusione dei modelli e delle rappresentazioni.
tests/	Test automatici per generatore, ontologia, reasoner e feature ML.

Appendice B – Riproducibilità e interpretazione dei risultati

```
python -m pip install -r requirements.txt
```

```
python -m src.main
```

```
python -m pytest
```

Per riprodurre gli esperimenti, installare le dipendenze indicate in requirements.txt dalla directory principale. Il comando di avvio apre il menu testuale; la suite pytest verifica i componenti implementati. L'esecuzione del reasoner richiede un ambiente Java compatibile con Owlready2/HermiT.